Northwind 2.0 Things You Should Know

Ala Armani

▶ *Applies To*

The content here may apply to Northwind 2.0 Developer Edition and Starter Edition.

Access VBA Reference

VBA (Visual Basic for Applications) is the programming language used in all Office Products. Learning VBA allows you to work with all Office products (not just Access).
When searching for "how-to", be sure to look for Access specific examples and include Microsoft Access in the search. Often solutions for the other Office products will work - but there is no guarantee. Microsoft Access is a mature product; that means there are a lot of examples out there; which is great for you!

It also means that older books on Access programming are still viable for you to look at. Many of the older books are still available on used book sites at a fraction of their original cost. Check the Microsoft website to determine what versions of Access are still being supported and go with those.

End of support resources for Office - Deploy Office | Microsoft Learn

Below are some links to Access documentation at Microsoft.

- Access Visual Basic for Applications (VBA) reference | Microsoft Learn
- VBA Glossary | Microsoft Learn
- Visual Basic conceptual topics | Microsoft Learn
- Visual Basic for Applications - Wikipedia
- VBA-Docs/API at main · MicrosoftDocs/VBA-Docs · GitHub

Trusted Locations and Enabled Content

Was this information helpful? Yes No

Microsoft Access files are Office files. Office files must be in a “Trusted Location” or have their “content enabled”. These items are considered “safe” because you created them, or they have come from a trustworthy source. Check for Trusted Locations occurs every time you open any office File. We will refer to this as Trusted/Enabled from here on. NOTE: If a new version of the application is released and opened from a non-trusted location the process of enabling the content will repeat.

Learn more about Trusted Locations.:

- Trusted Locations for Office files - Deploy Office | Microsoft Learn
- Decide whether to trust a database (Microsoft Support)
- Add, remove, or change a trusted location (Microsoft Support)

Macros, Functions and Subs

Macros, Functions and Subs are how you implement business logic into your Access database. It’s important for you to understand **Scope and Visibility** before your start.

- RunCode macro action | Microsoft Learn
- Introduction to macros | Microsoft Support
- Function statement (VBA) | Microsoft Learn
- Sub statement (VBA) | Microsoft Learn

Events (like clicking a control) on Controls on a form (e.g., buttons, text boxes, labels, etc.) trigger other processes, such as adding, deleting records, or opening forms. These processes can be implemented using either macros or VBA. Northwind Starter Edition uses mostly macros, and some VBA where macros are not able to perform needed functions. Northwind Developer Edition uses primarily VBA.

Some control types have built-in wizards to automatically create a macro. For example, adding a command button to a form opens a wizard that offers several choices of functionality for the button. Adding a combo box opens a wizard that can be configured to find a particular record on the form.

Navigation Pane

Was this information helpful?

The Navigation Pane is the main way you view and access all your database objects and it displays on the left side of the Access window by default.
The Northwind Navigation Pane has been customized. We created a custom category called **Northwind Starter 2.0**. This allows us to organize the objects by functional area.

- Use the Navigation Pane | Microsoft Support
- Customize the Navigation Pane | Microsoft Support

Scope and Visibility of Variables In VBA

It's important to learn about **Scope and Visibility** within Access/Office. You can start here:

- Understanding scope and visibility (VBA) | Microsoft Learn
- Public statement (VBA) | Microsoft Learn
- Private statement (VBA) | Microsoft Learn
- Static statement (VBA) | Microsoft Learn
- Understanding the lifetime of variables (VBA) | Microsoft Learn

Persisting Variables

Sometimes you need a variable to exist after the object which created it goes out of scope. See **Scope and Visibility** above. There are three primary ways to do this: Public Variables, TempVars and Storing the values in a local table. Many developers use a mix of these. Each has its pros and cons. More about each here:

VBA Module Public Variable:

- Declaring variables (VBA) | Microsoft Learn

TempVars:

- TempVars object (Access) | Microsoft Learn
- Power Tip: Maximize the use of TempVars in Access 2007 and 2010 | Microsoft 365 Blog

Was this information helpful?

Storing the values in local table

- Public variables and TempVars exist for the current session and go out of scope when the application is closed. But what if you want to keep user specific variables across sessions? You can store these types of values in a local table. In Northwind 2.0 one such variable is saved in a table that is called **SystemSettings**. The value in the table is **ShowWelcome**. This value tells Access if you want to see the Welcome screen each time you log in or not.

OpenArgs and StringFormat()

Developers often need to pass parameters from one form to another, or from a form to a report. These parameters convey important information, which the called function will then use to configure itself. There are several ways for the second form or report to get information from the first form. Here are a couple of those ways:

1. The second form can "look back" to the first form to pick up some values, possibly in either visible or invisible control. For example:
   `lngCustomerID = Forms!FirstForm!cboCustomerID`

2. The first form can save values to global variables or to *TempVars*. For example:
   `g_lngUserID = Me.cboUserID`
   TempVars.Add "UserID", Me.cboUserID

The method that is often used in Northwind Developer Edition as well as in our professional lives is using the OpenArgs argument of DoCmd.OpenForm or OpenReport. For example:

```
DoCmd.OpenForm "frmCompanyDetail", OpenArgs:=StringFormat("CompanyID=
{0} &CompanyTypeID={1}", Me.VendorID, ctVendor)
```

We are combining two techniques here: (1) the use of OpenArgs to pass in the VendorID and VendorType, and (2) the use of the *StringFormat()* function to create, for example, this string:

```
CompanyID=5&CompanyTypeID=2
```

This string looks very much like a query string as is used in a browser. It contains one or more "name/value pairs" separated by the ampersand character:

```
name1=value1&name2=value2
```

The advantage of such a string is that each value has a name. Compare this to a simpler approach where you would set OpenArgs only to "5,2". In such a case, it would take effort to

Was this information helpful?

find out what each value signifies. Naming each value makes the query string "self-describing" which is a good programming practice.

On the receiving end of *DoCmd.OpenForm* we are typically in the *Form_Open* or F*orm_Load* event and want to parse the *OpenArgs* string into its components.

In Northwind you can do this with the *StringToDictionary* function. It takes a querystring-like function and parses it into its components. These components are then stored in a *Scripting.Dictionary* object. Note that doing this requires you to use **Tools > References** and set a reference to **Microsoft Scripting Runtime** (scrrun.dll).

Features and benefits of the Dictionary object include these:

- The order of elements is not important
- Simple functions to add and remove elements of the collection
- Functions to loop over the collection, so you can know what is in it
- An *Exists* function so you can test if a certain element is available

Use of the dictionary object appears throughout Northwind. For example, the *Form_Load* event in *frmGenericDialog*.

Error Handling

Macros created with Control Wizards in Access seldom include error handling at all; VBA created with Control Wizards may be limited to a generic **MsgBox Err.Description**.

In Northwind 2.0 we show you how to do it better when using VBA code. We've implemented what's called a **Global Error Handler**. Errors that happen in any procedure call a function at the global level to show the error. The big advantage here is that error handling is consistent. And if the message needs to change (e.g., to additionally show the error number or to log the error to a file), it needs to be done in one place only.

clsErrorHandler is the Class Module that implements the error handling code. A class module keeps all its main and helper functions together in one unit, thus encapsulating the code.

The *AutoExec* macro calls the Startup function in *modStartup*. In Starter Edition, the function creates an instance of *clsErrorHandler* and saves it as a global variable that is available for use throughout the application. In Dev edition a static class is used – see the comments at the top of the class module.

In fact, the error handling code in procedures is so consistent that we were able to create all of it in less than five minutes using specific VBA code that outfitted each procedure with the proper error handler. (Code not included in the template). Both Northwind 2.0 Starter and

Was this information helpful?

Developer template editions were initially outfitted with this error handling approach.

IMPROVED ERROR HANDLING

Beginning with version 2.2 of Northwind Developer Edition, the error handler has been improved, thanks to feedback from the Access community. Starter edition is unchanged.

In essence, the error handler in the prior version (2.0 - released in April 2023) is:

```
Public Sub HandleError(…)
  MsgBox Err.Description
End Sub
```

In version 2.2 it is upgraded to:

```
Public Sub HandleError (…, Optional ByVal IsEventProcedure As Boolean
= False)
  If Not IsEventProcedure Then
    Err.Raise lngError, strErrSource
  End If
  MsgBox Err.Description
End Sub
```

To understand why this change was made, let's first understand what makes code run:

- The *AutoExec* macro calls the Startup procedure, which performs some initializations before opening the first form.
- The user interacts with the application, like opening a form or clicking a button, causing event procedures to fire such as **Form_Load** and **cmdPrintInvoice_Click**.

In addition to event procedures, applications have subroutines and functions--mostly in modules--and that code is called from the event procedures. These are called "standard" procedures.

In version 2.0 of Northwind, the standard procedures would handle their own errors with messages, but they would not somehow notify the calling event procedure that an error had occurred. This can be bad if the event procedure has subsequent code that should run regardless of the previous error handled by the called procedure. Sure, we could replace the subroutine with a function that returns success or failure, and code the event procedure accordingly, but that is not always an option.

In Northwind version 2.2, the standard procedures do not handle error messages, but rather, using Err.Raise, report them back to the calling event procedure. The calling event procedure

Was this information helpful?

then displays the raised error and resumes at **Exit_Handler**. This is better, because it allows the calling procedure to conclude gracefully.

To use the Northwind version 2.2 code, event procedures must pass into **HandleError** a third argument indicating that the caller is an event procedure. Northwind Dev Edition has been updated to do so.

An even more powerful error handler module would have support for “pushing and popping” procedures on a “stack” (array). The first element would always be the event procedure, so the extra argument is not needed. This implementation is beyond the goals of Northwind Dev Edition.

MRU List

MRU, or *Most Recently Used* is a list of recently used Orders and Purchase Orders. You may want to go back to these frequently to put them in the next Status. MRU lists are often seen in Office products as a list of recently used files that you may want to re-open again.

in Northwind Dev edition, to implement the MRU feature (which does not exist in the Starter edition) you must first establish the following items:

1. A table to store MRU information.
2. Code to update the table when an order or purchase order (PO) is opened.
3. Code to update the MRU dropdown in the ribbon.
4. Code to load the item when an MRU item is selected from the ribbon.

Let’s look at each of these in more detail.

1. Table to store MRU information.

The design of table MRU is worth reviewing, especially its indexes. Note that there is a duplicate index *SortIdx* to assist with quick sorting of the MRU items in the ribbon dropdown list, as well as a unique index to enforce the business rule that for each user an item can occur only once. For example, opening the same order twice does not create two records in the MRU table.

The table takes advantage of the fact that all MRU-related PK (primary key) fields in the database are AutoNumber, so the Long Integer data type can be used for *PKValue*.

2. Code to update the table when an order or P.O. is opened.

Was this information helpful?

In NW2 we chose to add to the MRU list only when a new record was created, not when an existing one was updated again. We certainly could move the *AddToMRU* call from *Form_AfterInsert* to *Form_AfterUpdate* to support that.

The *AddToMRU* and *DeleteFromMRU* procedures are implemented in *modGlobal*, which is a Standard Module whose public procedures are visible from any form.

AddToMRU (as the name suggests) adds the new item to the MRU table, and then optionally trims it back, deleting the oldest record, if it has grown beyond the max size (MAX_MRU_COUNT). The last step is probably the least well known to Access developers: the ribbon dropdown has to be refreshed and that is accomplished by calling InvalidateControl. This is a signal to the ribbon to re-run its initialization process.

3. Code to update MRU dropdown in the ribbon.

At startup time, and after *InvalidateControl* is called, a complex set of functions is executed to populate the ribbon. These procedures are called by the Ribbon XML in table *uSysRibbons* which says in part:

```
<group id="gCurrentStatus" label="MRU">
    <box id="bxMRU" boxStyle="vertical">
        <dropDown id="ddMRU"
                  getItemCount="ddMRU_GetItemCount"
                  getItemLabel="ddMRU_GetItemLabel"
                  getSelectedItemIndex="ddMRU_GetSelectedItemIndex"
                  getItemID="ddMRU_GetItemID"
                  onAction="ddMRU_OnAction"
                  screentip="Most Recently Used Objects">
        </dropDown>
    </box>
</group>
```

These four callback functions populate the dropdown. Note that this is very much the same idea as described herein for standard comboboxes.

If you uncomment the *Debug.Print* lines in *modRibbonCallback* and restart the application, the Immediate Window will present a sequence like this:

```
ddMRU_GetItemCount    ddMRU    6
ddMRU_GetItemLabel    ddMRU    0      Order 60, Proseware, Inc.
ddMRU_GetItemID       ddMRU    0       2
ddMRU_GetItemLabel    ddMRU    1      Order 62, Best For You Organics Company
ddMRU_GetItemID       ddMRU    1       4
ddMRU_GetItemLabel    ddMRU    2      Order 63, Wide World Importers
ddMRU_GetItemID       ddMRU    2       5
ddMRU_GetItemLabel    ddMRU    3      Order 66, Proseware, Inc.
ddMRU_GetItemID       ddMRU    3       8
ddMRU_GetItemLabel    ddMRU    4      Order 67, Best For You Organics Company
ddMRU_GetItemID       ddMRU    4       9
ddMRU_GetItemLabel    ddMRU    5      Order 68, Adatum Corporation
ddMRU_GetItemID       ddMRU    5       10
ddMRU_GetSelectedItemIndex ddMRU    0
```

Was this information helpful?

We can see here that Access is first calling a procedure that returns the number of items to load in the ByRef argument of *ddMRU_GetItemCount*. This is also the time when we open the query on MRU table and cache it because it is about to be used several times.

The ribbon then repeatedly calls two procedures to get the ID and Label values for the two-column dropdown.

Finally, it calls a procedure to disover which item should be selected. (In our case, it is the first one.)

4. Code for loading an item when the MRU item is selected from ribbon.

As with any other ribbon item, the *OnAction* property in the Ribbon XML specifies a callback function to be used to perform the action:

```
onAction="ddMRU_OnAction"
```

This procedure is implemented in *modRibbonCallback*. It re-uses the already open recordset to find the record with the selected item, then, depending on the TableName required, opens the corresponding form, passing in the PK value to be loaded.

Learn more

- Northwind 2.0 Developer Edition: Template-Tutorial
- Northwind 2.0 Developer Edition: All topics

Need more help?

How can we help you?

Want more options?

Was this information helpful?

Discover **Community**

Explore subscription benefits, browse training courses, learn how to secure your device, and more.



Microsoft 365 subscription benefits



Microsoft 365 training



Microsoft security



Accessibility center

What's new

Surface Pro

Surface Laptop

Surface Laptop Studio 2

Copilot for organizations

Copilot for personal use

AI in Windows

Explore Microsoft products

Windows 11 apps

Microsoft Store

Account profile

Download Center

Microsoft Store support

Returns

Order tracking

Certified Refurbished

Microsoft Store Promise

Flexible Payments

Education

Microsoft in education

Devices for education

Microsoft Teams for Education

Microsoft 365 Education

How to buy for your school

Educator training and development

Deals for students and parents

AI for education

Business

Microsoft AI

Microsoft Security

Dynamics 365

Microsoft 365

Microsoft Power Platform

Microsoft Teams

Microsoft 365 Copilot

Small Business

Developer & IT

Azure

Microsoft Developer

Microsoft Learn

Support for AI marketplace apps

Microsoft Tech Community

Microsoft Marketplace

Marketplace Rewards

Visual Studio

Company

Careers

About Microsoft

Company news

Privacy at Microsoft

Investors

Diversity and inclusion

Accessibility

Sustainability

Was this information helpful?

English (United States) Your Privacy Choices Consumer Health Privacy

Sitemap Contact Microsoft Privacy Terms of use Trademarks Safety & eco Recycling About our ads

© Microsoft 2026

Was this information helpful?